

REPORTES DE PROYECTO: ACTIVIDAD 3.2

ANALIZADOR SINTÁCTICO, SEMÁNTICO Y GENERACIÓN DE CÓDIGO INTERMEDIO EN LLVM IR

- **Asignatura:** Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 502)
- **Estudiante:** Daniel Flores Rojas
- **Matrícula:** A01737719
- **Institución:** Tecnológico de Monterrey
- **Fecha de Entrega:** 25 de mayo de 2026
- **Repositorio del Proyecto:** <https://github.com/danyflores7/Actividad-3.2.-Analizador.git>

1. INTRODUCCIÓN Y CONTRASTE DE ANALIZADORES SINTÁCTICOS

En la teoría de lenguajes y compiladores, la fase de análisis sintáctico representa el puente crítico entre la secuencia lineal de símbolos léxicos (tokens) y la estructura jerárquica que define la semántica de un programa. Esta validación se realiza mediante dos estrategias algorítmicas fundamentales:

Analizadores Sintácticos Top-Down (Descendentes)

- **Mecanismo:** Comienzan la evaluación desde el símbolo inicial de la gramática (la raíz del árbol) y descienden progresivamente intentando predecir las reglas de producción aplicables para emparejar los tokens de entrada (las hojas). Los algoritmos más comunes son los de descenso recursivo y los parsers $LL(k)$.
- **Vulnerabilidades:** Poseen una limitación estructural insalvable frente a la recursión por la izquierda. Si una gramática posee reglas de tipo $A \rightarrow A\alpha$, el analizador entrará en un ciclo infinito de llamadas recursivas antes de consumir cualquier token. Para solventar esto, se requiere transformar la gramática original

mediante algoritmos de eliminación de recursión e introducción de factorización izquierda, lo que a menudo altera la legibilidad e intuitividad del diseño gramatical.

Analizadores Sintácticos Bottom-Up (Ascendentes)

- **Mecanismo:** Comienzan desde los tokens del código fuente (las hojas del árbol) y acumulan elementos en una pila semántica mediante operaciones de desplazamiento (*shift*). Cuando el subconjunto de elementos en el tope de la pila coincide con el lado derecho de una regla de producción, se ejecuta una reducción (*reduce*), sustituyendo dichos elementos por el no terminal correspondiente (la raíz). Los algoritmos más representativos son los parsers de tipo LR(k) y LALR(1).
- **Ventajas:** Manejan de forma nativa e intuitiva la recursión por la izquierda. Esto facilita enormemente el diseño de listas de sentencias, listas de declaraciones y expresiones matemáticas sin peligro de desbordamiento de pila en el parser.

Enfoque Adoptado en la Solución

Este proyecto utiliza la biblioteca **PLY (Python Lex-Yacc)**, la cual construye un motor de análisis sintáctico **Bottom-Up de tipo LALR(1)**. Esta elección fue fundamental para estructurar el compilador de la extensión Clite. Permite definir la acumulación de múltiples sentencias y declaraciones consecutivas de forma lineal e iterativa a través de reglas recursivas por la izquierda, garantizando un consumo mínimo de memoria en la pila del parser y previniendo conflictos sintácticos complejos durante la construcción del Árbol de Sintaxis Abstracta (AST).

2. ARQUITECTURA DEL COMPILADOR Y ESTRUCTURA POR ETAPAS

El compilador desarrollado se compone de cuatro etapas modulares integradas mediante el patrón de diseño estructural **Visitor**, aislando la validación sintáctica de la lógica de emisión de código:

Código Fuente (.c) -> [Analizador Léxico] -> [Analizador Sintáctico] -> [AST Nodes] ->

1. **Analizador Léxico (Lexer):** Implementado en `analisis.py` utilizando el submódulo `ply.lex`. Transforma el flujo de caracteres del archivo fuente en un flujo ordenado de unidades lógicas con significado (tokens). Reconoce palabras reservadas como `while`, `if`, `else`, `return`, `for`, `do`, `switch`, `case` y `default`, además de identificar variables (`ID`), constantes enteras (`INTLIT`) y constantes de punto flotante (`FLOATLIT`).

2. **Analizador Sintáctico (Parser):** Diseñado con `ply.yacc` . Implementa la gramática formal del subconjunto de C (Clite) extendida con llamadas a funciones, múltiples parámetros y estructuras de control avanzadas. A medida que procesa las reducciones de los tokens, instancia los nodos correspondientes de la jerarquía de clases de sintaxis abstracta.
3. **Árbol de Sintaxis Abstracta (AST):** Definido en `arbol.py` . Modelado mediante clases orientadas a objetos que heredan de la clase abstracta `ASTNode` . Los nodos, como `WhileNode` , `ForNode` , `SwitchNode` o `FunctionNode` , no ejecutan acciones por sí mismos; en su lugar, almacenan de forma segura las referencias estructurales del código fuente y exponen un método común `accept(visitor)` que actúa como puerta de enlace para que agentes externos operen sobre ellos.
4. **Generador de Código Intermedio (IRGenerator):** Implementado en `analisis.py` como un visitante que hereda de la clase base abstracta `Visitor` . Recorre el AST nodo por nodo, gestiona una pila semántica interna de operandos y utiliza la biblioteca **LLVMLite** para emitir instrucciones SSA de código intermedio nativo de LLVM.

3. ANÁLISIS DE CÓDIGO CLAVE Y GENERACIÓN DE MEMORIA EN LLVM IR

A. Manejo de Memoria Local mediante `alloca` , `store` y `load`

En la arquitectura SSA (Static Single Assignment) de LLVM IR, los registros temporales virtuales son inmutables: una vez asignados, su valor no puede volver a cambiar. Para dar soporte a las variables locales mutables en nuestro lenguaje de programación, es indispensable reservar almacenamiento real dentro de la pila (*stack frame*) de la función activa mediante la instrucción `alloca` .

Esto se implementa de manera coordinada dentro de los métodos `visit_variable` y `visit_assignment` de la clase `IRGenerator` :

```
def visit_variable(self, node: Variable):
    if node.name not in self.symbol_table:
        # Declaración: se reserva memoria física en la pila
        v_type = node.type
        llvm_type = floatType if v_type == 'float' else intType
        ptr = self.builder.alloca(llvm_type, name=node.name)
        self.symbol_table[node.name] = ptr
```

```

        self.variable_types[node.name] = v_type
    else:
        # Referencia: se lee el valor actual de memoria y se empuja a la pila
        ptr = self.symbol_table[node.name]
        val = self.builder.load(ptr, name=node.name)
        self.stack.append(val)

```

Cuando se compila una asignación (por ejemplo, `y = y + 1;`), se evalúa primero el lado derecho (que realiza una lectura `load` de `y`, le suma la constante y deja el resultado en la pila del compilador) y luego se escribe dicho valor de regreso en el puntero guardado de la variable en la pila usando un `store`:

```

def visit_assignment(self, node: AssignmentNode):
    node.expr.accept(self)
    val = self.stack.pop()
    ptr = self.symbol_table[node.var_name]
    var_type = self.variable_types.get(node.var_name, 'int')

    # Conversión implícita de tipos si existe discrepancia
    if var_type == 'float' and val.type == intType:
        val = self.builder.sitofp(val, floatType)
    elif var_type == 'int' and val.type == floatType:
        val = self.builder.fptosi(val, intType)

    self.builder.store(val, ptr)

```

B. Coordinación de Estructuras de Control Avanzadas (`FOR` , `DO/WHILE` y `SWITCH`)

La bifurcación y repetición lógica de las instrucciones se implementa mediante la creación dinámica de bloques básicos (`Basic Blocks`) de código y saltos condicionales (`cbranch`) e incondicionales (`branch`).

- **Ciclo FOR:** Se modela instanciando tres bloques básicos: `for-head` , `for-body` y `for-exit` . Primero se ejecuta la instrucción de inicialización (`init_stmt`). Luego, el flujo salta incondicionalmente al encabezado para evaluar la expresión condicional. Si la evaluación de la condición es verdadera, se ingresa a `for-body` , donde al término del cuerpo principal del ciclo se ejecuta la sentencia de

incremento (`incr_stmt`) antes de realizar un salto incondicional de regreso al encabezado.

- **Ciclo DO/WHILE:** A diferencia del `while` tradicional, esta estructura garantiza que el cuerpo de ejecución se procese al menos una vez antes de verificar la expresión de control. Por ello, el compilador genera un salto incondicional directo hacia el bloque `do-body` . Al final del bloque, se genera el código de evaluación de la condición lógica y se aplica un `cbranch` . Si la condición evalúa como verdadera, el flujo salta de regreso a `do-body` , de lo contrario, escapa al bloque secuencial de salida `do-exit` .
- **Estructura SWITCH:** Implementada mapeando la instrucción nativa `self.builder.switch(cond_val, default_block)` de LLVM. Se crea un bloque de salida común `switch-exit` y un bloque `case-default` (que apunta a `switch-exit` si el usuario no especificó una alternativa por defecto). Posteriormente, se itera sobre los casos constantes y se añaden de forma indexada como ramas de la tabla de saltos del switch usando `switch_inst.add_case(const_val, case_block)` . Al final del cuerpo de cada caso, se emite de forma obligatoria un salto incondicional a `switch-exit` para desactivar el comportamiento de desbordamiento de casos (*fall-through*).

C. Conversión Implícita de Tipos (`sitofp` y `fptosi`)

En LLVM, las operaciones matemáticas y de asignación directa son estrictamente homogéneas. El compilador resuelve la mezcla de enteros (`int`) y números de punto flotante (`float`) interceptando las asignaciones. Si se intenta escribir un entero en una variable de tipo flotante, el compilador genera la instrucción de promoción de signo de bits `sitofp` . Si se intenta almacenar un flotante en una variable entera, genera la instrucción `fptosi` , la cual trunca matemáticamente los decimales del valor flotante hacia cero para ajustarlo en el formato del entero de 32 bits de forma segura.

D. Soporte y Enlace Variádico con la Función Externa `printf`

Para permitir que los programas impriman valores numéricos reales en la terminal, se interceptan las llamadas sintácticas dirigidas a la palabra reservada `"printf"` . En ese momento, se inyecta la declaración externa de la biblioteca estándar de C (`declare i32 @printf(i8*, ...)`), declarada con el atributo variádico `var_arg=True` para soportar argumentos dinámicos indeterminados.

Asimismo, se introduce en el módulo una cadena global única que almacena la secuencia de formato `%d\n\0` . Al compilar la llamada, la dirección de memoria de esta variable global (que posee un tipo estricto de puntero de arreglo `[4 x i8]*`) se transforma mediante una instrucción de casting directo a nivel de bits (`bitcast`) a un puntero

genérico a caracteres (`i8*`). Esta dirección transformada se suministra como el primer parámetro de `printf` , logrando la interoperabilidad directa con `libc` .

4. MANUAL DE USUARIO E INSTRUCCIONES DE EJECUCIÓN

Requisitos de Software

Para ejecutar este compilador y su suite de verificación, el entorno debe contar con:

1. **Python 3.8** o superior instalado.
2. La biblioteca de análisis léxico y sintáctico **PLY**.
3. La biblioteca de generación y validación de IR de LLVM **llvmlite**.

Instalación de las Dependencias

Ejecuta el siguiente comando en la consola de tu sistema dentro del entorno virtual de trabajo activo:

```
pip install ply llvmlite
```

Instrucciones para Compilar y Validar

- **Ejecutar el Compilador de Prueba Base (`analisis.py`)**: Este script compila el código por defecto configurado directamente en el archivo y despliega en la salida estándar el Árbol sintáctico (AST), el contenido de la pila semántica y el módulo de LLVM IR generado.

```
python analisis.py
```

- **Ejecutar el Suite de Pruebas Automatizado (`verificar_pruebas.py`)**: Este script ejecuta un lote de pruebas leyendo dinámicamente los 5 archivos fuente ubicados en el directorio `pruebas/` , validando que cada uno compile sin fallas y mostrando en pantalla un mensaje claro de éxito junto al código intermedio de LLVM de cada uno.

```
python verificar_pruebas.py
```

5. CATÁLOGO DE PRUEBAS Y RESULTADOS DEL MÓDULO LLVM IR

A continuación, se adjunta la compilación exacta y el módulo LLVM IR generado por el compilador para cada uno de los 5 programas de prueba obligatorios requeridos por la rúbrica de evaluación:

PROGRAMA 1: Algoritmo de Factorial Recursivo (Recursión Simple)

- **Código de Entrada (pruebas/programa1_factorial.c):**

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    printf(factorial(5));
    return 0;
}
```

- **Código Intermedio Generado en LLVM IR:**

```
; ModuleID = "prog"
target triple = "unknown-unknown-unknown"
target datalayout = ""

define i32 @"factorial"(i32 %"n")
{
entry:
    %"n.1" = alloca i32
    store i32 %"n", i32* %"n.1"
    %"n.2" = load i32, i32* %"n.1"
    %".4" = icmp sle i32 %"n.2", 1
    %".5" = zext i1 %".4" to i32
    %".6" = icmp ne i32 %".5", 0
    br i1 %".6", label %"if-then", label %"if-exit"
if-then:
    ret i32 1
```

```

if-exit:
    %".3" = load i32, i32* %".1"
    %".4" = load i32, i32* %".1"
    %".9" = sub i32 %".4", 1
    %".10" = call i32 @"factorial"(i32 %".9")
    %".11" = mul i32 %".3", %".10"
    ret i32 %".11"
}

define i32 @"main"()
{
entry:
    %".2" = call i32 @"factorial"(i32 5)
    %".3" = bitcast [4 x i8]* @"fstr_3" to i8*
    %".4" = call i32 (i8*, ...) @"printf"(i8* %".3", i32 %".2")
    ret i32 0
}

declare i32 @"printf"(i8* %".1", ...)

@"fstr_3" = internal constant [4 x i8] c"%d\0a\00"

```

PROGRAMA 2: Sucesión de Fibonacci (Doble Recursión)

- Código de Entrada (`pruebas/programa2_fibonacci.c`):

```

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    printf(fibonacci(6));
    return 0;
}

```


- **Código Intermedio Generado en LLVM IR:**

```
; ModuleID = "prog"
target triple = "unknown-unknown-unknown"
target datalayout = ""

define i32 @"fibonacci"(i32 %"n")
{
entry:
    %"n.1" = alloca i32
    store i32 %"n", i32* %"n.1"
    %"n.2" = load i32, i32* %"n.1"
    %".4" = icmp sle i32 %"n.2", 1
    %".5" = zext i1 %".4" to i32
    %".6" = icmp ne i32 %".5", 0
    br i1 %".6", label %"if-then", label %"if-exit"
if-then:
    %"n.3" = load i32, i32* %"n.1"
    ret i32 %"n.3"
if-exit:
    %"n.4" = load i32, i32* %"n.1"
    %".9" = sub i32 %"n.4", 1
    %".10" = call i32 @"fibonacci"(i32 %".9")
    %"n.5" = load i32, i32* %"n.1"
    %".11" = sub i32 %"n.5", 2
    %".12" = call i32 @"fibonacci"(i32 %".11")
    %".13" = add i32 %".10", %".12"
    ret i32 %".13"
}

define i32 @"main"()
{
entry:
    %".2" = call i32 @"fibonacci"(i32 6)
    %".3" = bitcast [4 x i8]* @"fstr_3" to i8*
    %".4" = call i32 (i8*, ...) @"printf"(i8* %".3", i32 %".2")
    ret i32 0
}

declare i32 @"printf"(i8* %".1", ...)
```

```
@"fstr_3" = internal constant [4 x i8] c"%d\0a\00"
```

PROGRAMA 3: Algoritmo de Potencia (Recursión con Parámetros Múltiples)

- **Código de Entrada (pruebas/programa3_potencia.c):**

```
int potencia(int base, int exp) {
    if (exp <= 0) {
        return 1;
    }
    return base * potencia(base, exp - 1);
}

int main() {
    printf(potencia(2, 4));
    return 0;
}
```

- **Código Intermedio Generado en LLVM IR:**

```
; ModuleID = "prog"
target triple = "unknown-unknown-unknown"
target datalayout = ""

define i32 @"potencia"(i32 %"base", i32 %"exp")
{
entry:
    %"base.1" = alloca i32
    store i32 %"base", i32* %"base.1"
    %"exp.1" = alloca i32
    store i32 %"exp", i32* %"exp.1"
    %"exp.2" = load i32, i32* %"exp.1"
    %".6" = icmp sle i32 %"exp.2", 0
    %".7" = zext i1 %".6" to i32
    %".8" = icmp ne i32 %".7", 0
    br i1 %".8", label %"if-then", label %"if-exit"
if-then:
```

```

    ret i32 1
if-exit:
    %"base.2" = load i32, i32* %"base.1"
    %"base.3" = load i32, i32* %"base.1"
    %"exp.3" = load i32, i32* %"exp.1"
    %".11" = sub i32 %"exp.3", 1
    %".12" = call i32 @"potencia"(i32 %"base.3", i32 %".11")
    %".13" = mul i32 %"base.2", %".12"
    ret i32 %".13"
}

define i32 @"main"()
{
entry:
    %".2" = call i32 @"potencia"(i32 2, i32 4)
    %".3" = bitcast [4 x i8]* @"fstr_3" to i8*
    %".4" = call i32 (i8*, ...) @"printf"(i8* %".3", i32 %".2")
    ret i32 0
}

declare i32 @"printf"(i8* %".1", ...)

@"fstr_3" = internal constant [4 x i8] c"%d\0a\00"

```

PROGRAMA 4: Ciclo FOR Acumulativo

- Código de Entrada (`pruebas/programa4_ciclo_for.c`):

```

int main() {
    int i;
    int cuenta;
    cuenta = 0;
    for (i = 0; i < 5; i = i + 1) {
        cuenta = cuenta + i;
    }
    printf(cuenta);
    return 0;
}

```

- **Código Intermedio Generado en LLVM IR:**

```
; ModuleID = "prog"
target triple = "unknown-unknown-unknown"
target datalayout = ""

define i32 @"main"()
{
entry:
  %"i" = alloca i32
  %"cuenta" = alloca i32
  store i32 0, i32* %"cuenta"
  store i32 0, i32* %"i"
  br label %"for-head"
for-head:
  %"i.1" = load i32, i32* %"i"
  %".5" = icmp slt i32 %"i.1", 5
  %".6" = zext i1 %".5" to i32
  %".7" = icmp ne i32 %".6", 0
  br i1 %".7", label %"for-body", label %"for-exit"
for-body:
  %"cuenta.1" = load i32, i32* %"cuenta"
  %"i.2" = load i32, i32* %"i"
  %".9" = add i32 %"cuenta.1", %"i.2"
  store i32 %".9", i32* %"cuenta"
  %"i.3" = load i32, i32* %"i"
  %".11" = add i32 %"i.3", 1
  store i32 %".11", i32* %"i"
  br label %"for-head"
for-exit:
  %"cuenta.2" = load i32, i32* %"cuenta"
  %".14" = bitcast [4 x i8]* @"fstr_2" to i8*
  %".15" = call i32 (i8*, ...) @"printf"(i8* %".14", i32 %"cuenta.2")
  ret i32 0
}

declare i32 @"printf"(i8* %".1", ...)

@"fstr_2" = internal constant [4 x i8] c"%d\0a\00"
```

PROGRAMA 5: Asignación Heterogénea (Conversión Implícita)

- Código de Entrada (`pruebas/programa5_conversion_tipos.c`):

```
int main() {  
    float x;  
    int y;  
    y = 10;  
    x = y;  
    printf(y);  
    return 0;  
}
```

- Código Intermedio Generado en LLVM IR:

```
; ModuleID = "prog"  
target triple = "unknown-unknown-unknown"  
target datalayout = ""  
  
define i32 @"main"()  
{  
entry:  
    %"x" = alloca float  
    %"y" = alloca i32  
    store i32 10, i32* %"y"  
    %"y.1" = load i32, i32* %"y"  
    %".3" = sitofp i32 %"y.1" to float  
    store float %".3", float* %"x"  
    %"y.2" = load i32, i32* %"y"  
    %".5" = bitcast [4 x i8]* @"fstr_2" to i8*  
    %".6" = call i32 (i8*, ...) @"printf"(i8* %".5", i32 %"y.2")  
    ret i32 0  
}  
  
declare i32 @"printf"(i8* %".1", ...)  
  
@"fstr_2" = internal constant [4 x i8] c"%d\0a\00"
```

6. REFLEXIÓN PERSONAL DEL APRENDIZAJE

El desarrollo e implementación completa de este compilador representó una de las experiencias educativas y técnicas más desafiantes de mi trayectoria académica. A lo largo de la actividad, logré unificar la teoría gramatical libre de contexto y los autómatas finitos con aspectos prácticos de optimización e ingeniería de software a bajo nivel.

Trabajar directamente sobre la infraestructura SSA de LLVM redefinió mi comprensión sobre el flujo de ejecución en una CPU. Mapear un lenguaje imperativo clásico que permite modificar variables libremente a una arquitectura donde las asignaciones lógicas a registros virtuales son estrictamente constantes me ayudó la administración física de la memoria del marco de pila mediante asignaciones con `alloca` e instrucciones explícitas de carga y guardado (`load` y `store`). Del mismo modo, resolver la discrepancia de tipos de datos de forma silenciosa forzando instrucciones como `sitofp` y `fptosi` me enseñó a valorar las inmensas capas de abstracción y seguridad semántica que los lenguajes modernos nos proveen día con día.

Finalmente, diseñar los saltos condicionales y controlar el enlazado lógico de bloques de control (como la tabla de saltos del `SWITCH` o el incremento tardío del `FOR`) mejoró mis habilidades de depuración de bajo nivel y diseño arquitectónico. Este proyecto ha transformado definitivamente mi perspectiva técnica como desarrollador de software.

7. CONTROL DE VERSIONES DEL PROYECTO (GIT)

El desarrollo progresivo, la implementación modular de cada regla sintáctica de Clite y la adición secuencial de bloques de código en el backend de LLVM se llevaron a cabo utilizando el estándar de **un commit por regla de producción**. El historial de cambios se encuentra disponible públicamente en la siguiente liga:

Repositorio de GitHub: <https://github.com/danyflores7/Actividad-3.2.-Analizador.git>